

第 26 章：OOP: The Big Picture (OOP 全景图)

原书：《Learning Python》（6th Edition），作者 Mark Lutz 本章地位：本书面向对象编程（OOP）部分的"开篇全景图"。前三部分（第 4、10、16、17 章）打好了函数、作用域、闭包等基础，本章不教具体语法，而是先用高空视角讲清 OOP 为什么存在（代码复用）、类与实例的关系（蓝图 vs 具体）、属性查找机制（继承搜索）三大问题，为第 27 章开始的语法细节铺路。

26.1 开篇：从"对象"到"面向对象"

英文原文

So far in this book, we've been using the term "object" generically. Really, the code written up to this point has been object-based—we've passed objects around our scripts, used them in expressions, called their methods, and so on. For our code to qualify as being truly object-oriented (OO), though, our objects will generally need to also participate in something called an inheritance hierarchy. This chapter begins our exploration of the Python class—a coding structure and device used to implement new kinds of objects in Python.

on that support inheritance. Classes are Python's main object-oriented programming (OOP) tool, so we'll also study OOP basics along the way in this part of the book. OOP offers a different and often more effective way of programming. Like functions, we can use classes to factor code to minimize redundancy. Unlike functions, classes make it easy to write new programs by customizing existing code instead of changing it in place. In Python, classes are created with a new statement: the class. As you'll see, the objects defined with classes can look a lot like the built-in object types we employed earlier in the book. In fact, classes really just apply and extend the ideas we've already covered; roughly, they are packages of functions that use and process built-in objects. Classes, though, are designed to create and manage new objects, and support inheritance—a mechanism of code customization and reuse above and beyond anything we've seen so far. One note up front: in Python, OOP is entirely optional, and you don't need to use classes just to get started. You can get plenty of work done with simpler constructs such as functions, or even simple top-level script code. Because using classes well requires some up-front planning, they tend to be of more interest to people who work in strategic mode (doing long-term product development) than to people who work in tactical mode (where time is in very short supply). Still, as you'll see in this part of the book, classes turn out to

be one of the most useful tools Python provides. When used well, classes can actually cut development time radically. They're also employed in popular Python libraries, so most Python programmers will usually find at least a working knowledge of class basics helpful.

中文翻译

到目前为止，本书一直在泛泛地使用“对象”（object）这个词。实际上，截至现在的代码都是基于对象（object-based）的——我们在脚本中传递对象、在表达式中使用它们、调用它们的方法，等等。不过，要让我们的代码真正算得上面向对象（object-oriented，OO），我们的对象通常还需要参与一种叫做继承层次（inheritance hierarchy）的东西。本章开始探索 Python 的类（class）——一种用于在 Python 中实现支持继承的新类型对象的编码结构和机制。类（class）是 Python 主要的面向对象编程（OOP）工具，因此我们也会在本书这一部分顺带学习 OOP 基础。OOP 提供了一种不同的、往往更有效的编程方式。和函数一样，我们可以用类来分解（factor）代码以最小化冗余（redundancy）。与函数不同的是，类让我们可以轻松地通过定制（customizing）现有代码来编写新程序，而不是就地修改它。在 Python 中，类通过一个新语句创建：class 语句。你将会看到，用类定义的对象看起来和本书前面用到的内置对象类型非常相似。事实上，类只是应用和扩展了我们已经学过的思想；粗略地说，它们是打包了使用并处理内置对象的函数的“包裹”（packages of functions）。不过，类的设计目的是创建和管理新对象，并支持继承（inheritance）。

nance) ——一种超越我们目前所见一切机制的代码定制与复用机制。先说一句：在 Python 中，OOP 完全是可选的，你不需要为了起步而使用类。用函数之类的更简单结构，甚至直接用顶层脚本代码，你也能完成大量工作。因为用好类需要一些前置规划，它们往往对战略 (strategic) 模式工作的人 (做长期产品开发) 比对战术 (tactical) 模式工作的人 (时间极其紧张) 更有吸引力。不过，正如你在本书这一部分将看到的，类最终被证明是 Python 提供的最有用的工具之一。用得好时，类能真正大幅缩短开发时间。流行的 Python 库也都在使用类，所以大多数 Python 程序员通常都会发现，至少掌握类的基础知识是有帮助的。

深度理解

·核心概念：本章引入全书最重要的分界线——“基于对象”与“面向对象”不是一回事。之前的代码到处在用对象 (列表、字典、字符串……)，但那只是“使用对象”；真正的 OO 要求对象参与继承层次，即对象之间存在“上下级”关系，下级能自动获得上级的属性。

·底层实现：在 CPython 中，类本身也是运行时对象 (type 的实例)，存储在堆上；class 语句执行时会创建类对象并绑定到名字。实例对象与类对象之间、类对象与超类对象之间都存在指针链接，这就是后面属性搜索要爬的“树”。

·设计原因：为什么需要类？两条路：一是分解冗余 (和函数一样)，二是定制而非修改——这是函数做不到的。修改现有代码有风险 (可能破坏已工作的功能)，而“在不动原代码的前提下扩展它”才是工程上最安全、最省力的演进

方式。生活例子：给手机加保护壳、贴膜，是"定制"；拆开手机换主板，是"修改"。前者随时可逆，后者风险高。

·解决什么实际问题：大型项目反复造轮子、重复代码泛滥；改一处逻辑要牵动几十处调用。类的继承让"通用逻辑写一次、各子类按需微调"成为可能。

·初学者误区：① 以为"代码里出现 class 就算 OOP"——没有继承关系参与的类用处有限；② 以为"学 Python 必须学 OOP 才能写程序"——本章明确说 OOP 完全可选，脚本小工具用函数足够，战略级项目才值得上类；③ 以为类能凭空造出新东西——实际上类内部本质还是"包装函数 + 处理内置类型"。

26.2 为什么使用类？ (Why Use Classes?)

英文原文

Remember when this book told you that programs "do things with stuff" in Chapters 4 and 10? In simple terms, classes are just a way to define new sorts of stuff, reflecting real objects in a program's domain. For instance, suppose we decide to implement that hypothetical pizza-making robot we used as an example in Chapter 16. If we implement it using classes, we can model more of its real-world structure and relationships. Two aspects of OOP could be useful here: Inheritance — Pizza-making robots are kinds of robots, so they possess the usual robot-y properties. In OOP ter

ms, we say they "inherit" properties from the general category of all robots. These common properties need to be implemented only once for the general case and can be reused in part or in full by all types of robots we may build in the future. Composition — Pizza-making robots are really collections of components that work together as a team. For instance, for our robot to be successful, it might need arms to roll dough, motors to maneuver to the oven, and so on. In OOP parlance, our robot is an example of composition; it contains other objects that it activates to do its bidding. Each component might be coded as a class, which defines its own behavior and relationships. While you may never build pizza-making robots, general OOP ideas like inheritance and composition apply to any application that can be decomposed into a set of objects. For example, in typical GUI systems, interfaces are written as collections of widgets—buttons, labels, and so on—which are all drawn when their container is drawn (composition). Moreover, we may be able to write our own custom widgets—buttons with unique fonts, labels with new color schemes, and the like—which are specialized versions of more general interface devices (inheritance). From a more concrete programming perspective, classes are Python program units, just like functions and modules: they are another compartment for packaging logic and data. In fact, classes also define new namespaces, much like modu

les. But, compared to other program units we've already seen, classes have three critical distinctions that make them more useful when it comes to building new objects: Multiple instances — Classes are essentially factories for generating one or more objects. Every time we call a class, we generate a new object with a distinct namespace. Each object generated from a class has access to the class's attributes and gets a namespace of its own for data that varies per object. This is similar to the per-call state retention of Chapter 17's closure functions, but is explicit and natural in classes, and is just one of the things that classes do. Classes offer a more complete programming solution. Customization via inheritance — Classes also support the OOP notion of inheritance: we can extend a class by redefining its attributes outside the class itself in new software components coded as subclasses. More generally, classes can build up namespace hierarchies, which define names to be used by objects created from classes in the hierarchy. This supports multiple customizable behaviors more directly than other tools. Operator overloading — By providing special protocol methods, classes can define objects that respond to the sorts of operations we saw at work on built-in types. For instance, objects made with classes can be sliced, concatenated, indexed, and so on. Python provides hooks that classes can use to intercept and implement any built-in type operation. At its base, the m

mechanism of OOP in Python largely boils down to just two bits of magic: a special first argument in functions (to receive the subject of a call) and inheritance attribute search (to support programming by customization). Other than this, the model is largely just functions that ultimately process built-in types. While not radically new, though, OOP adds an extra layer of structure that supports programming better than flat procedural models. Along with the functional tools we met earlier, it represents a major abstraction step above computer hardware that helps us build more sophisticated programs.

中文翻译

还记得本书第 4 章和第 10 章告诉你的“程序就是‘用东西做事’”吗？简单来说，类只是定义新种类“东西”（stuff）的一种方式，用来反映程序领域中真实存在的对象。例如，假设我们决定实现第 16 章举例用的那个想象中的披萨制作机器人。如果我们用类来实现它，就能更贴近它现实世界的结构和关系。OOP 的两个方面在这里很有用：继承（Inheritance）——披萨制作机器人是机器人（robot）的一种，因此它们具备机器人通常的属性。用 OOP 的说法，我们说它们从“所有机器人”这一通用类别中“继承”（inherit）属性。这些通用属性只需要为通用情况实现一次，就能被我们未来可能建造的所有类型的机器人部分或全部复用。组合（Composition）——披萨制作机器人实际上是一组协同工作的组件集合。例如，要让我们的机器人成功工作，它可能需要手臂来擀面团、需要电机来移动到烤箱前，等等。用 OOP 的

行话来说，我们的机器人是组合（composition）的一个例子：它包含其他对象，并激活这些对象来执行它的指令。每个组件都可以编写成一个类，定义它自己的行为 and 关系。虽然你可能永远不会造披萨机器人，但继承和组合这类通用 OOP 思想适用于任何可以分解为一组对象的应用程序。例如，在典型的 GUI（图形用户界面）系统中，界面被写成一组组件（widget）——按钮、标签等——当容器被绘制时它们全部被绘制（组合）。此外，我们或许还能编写自己的自定义组件——特殊字体的按钮、新配色方案的标签等——它们是更通用界面设备的专门化版本（继承）。从更具体的编程角度看，类（class）是 Python 的程序单元（program units），就像函数和模块一样：它们是打包逻辑和数据的又一个隔间。事实上，类也定义新的命名空间（namespace），很像模块。但与我们已经见过的其他程序单元相比，类有三个关键区别，使它们在构建新对象时更有用：多实例（Multiple instances）——类本质上是生成一个或多个对象的工厂（factories）。每次我们调用一个类，就会生成一个新对象，它拥有一个独立的命名空间。从类生成的每个对象都能访问类的属性，并且拥有自己的命名空间来存放因对象而异的属性数据。这类似于第 17 章闭包（closure）函数的每次调用状态保持，但在类中是显式且自然的，而且这只是类的功能之一。类提供了更完整的编程解决方案。通过继承定制（Customization via inheritance）——类还支持 OOP 的继承概念：我们可以通过在新软件组件（编写为子类（subclass））中重新定义类的属性来扩展（extend）这个类。更一般地说，类可以构建命名空间层次（namespace hierarchies），它定义了由该层次中类创建的对象将使用的名字。这比其他工具更直接地支持多种可定制的

行为。运算符重载 (Operator overloading) ——通过提供特殊的协议方法 (protocol methods)，类可以定义出能响应我们之前在内置类型上看到过的那些运算 (operations) 的对象。例如，用类创建的对象可以被切片、拼接、索引等。Python 提供了挂钩 (hooks)，类可以用它来拦截并实现任何内置类型的运算。从根本上说，Python 中 OOP 的机制很大程度上归结为两件“魔法”：函数中一个特殊的第一个参数（用于接收调用主体）和继承属性搜索（用于支持通过定制编程）。除此之外，该模型在很大程度上只是最终处理内置类型的函数。虽然并非革命性创新，但 OOP 增加了一层额外的结构，比扁平的面向过程模型更能支持编程。与我们之前遇到的函数式工具一起，它代表了计算机硬件之上的一次重大抽象跃升，帮助我们构建更复杂的程序。

深度理解

·核心概念：本小节是全章的核心——类的三大超能力：多实例、继承定制、运算符重载。以及贯穿全书的结论：Python 的 OOP 只有两件魔法——特殊的第一参数 + 继承属性搜索。

·底层实现：“类是工厂”对应到 CPython：C1() 调用触发类型对象的 call，内部完成实例内存分配 (new) 并初始化 (init)。每次调用都是独立的内存块、独立的 dict 命名空间。而“继承”对应类对象里指向超类的元组 (bases)，属性搜索就是沿这条链向上爬。

·设计原因：为什么把“特殊第一参数 + 属性树搜索”作为全部机制？因为简单。Lutz 反复强调：OOP 不需要魔法，只需要两条简单规则，其余都是普通函数处理内建类型

。这符合 Python "显式优于隐式"的哲学——复杂度被压到最低，机制就可预测、可调试。

·解决什么实际问题：① 披萨机器人的"手臂/电机"= 组合：一个对象包含其他对象，各自是独立的类；② GUI 的"容器绘制时子控件跟着绘制"= 组合；"特殊字体的按钮"= 继承。这两大模式几乎能覆盖所有可分解为对象的应用。

·初学者误区：① 混淆"继承"与"组合"——继承是"is-a"（披萨机器人是机器人），组合是"has-a"（机器人有手臂）。判断口诀：能说"是"用继承，能说"有"用组合。② 以为运算符重载是炫技功能——它真正的价值是让自定义对象"看起来像内置类型"，从而无缝融入既有代码。③ 以为 OOP 机制复杂——本章预告：Python 的 OOP 就这么点东西，别自己吓自己。

26.3 OOP 从 30,000 英尺高空俯瞰 (OOP from 30,000 Feet)

英文原文

Before we dig into what this all means in terms of code, let's get a better handle on the general ideas behind OOP. If you've never done anything object-oriented in your life before now, some of the terminology in this chapter may seem a bit perplexing on the first pass. Moreover, the motivation for these terms may be elusive until you've had a chance to study the ways that programmers apply them in larger systems. O

OP is as much an experience as a technology.

中文翻译

在我们深入探讨这一切在代码层面意味着什么之前，先让我们更好地把握 OOP 背后的一般思想。如果你以前从未做过任何面向对象的开发，本章的一些术语在第一遍阅读时可能会显得有些令人困惑。而且，在你有机会研究程序员在更大系统中应用这些术语的方式之前，这些术语的动机可能难以捉摸。OOP 与其说是一门技术，不如说是一种经验。

深度理解

- 核心概念：这是作者的"免责声明"——先别急着抠术语，30,000 英尺高空看全景。OOP 术语（类、实例、继承、多态）的价值要在大系统中才显现，单看语法无法体会。
- 底层实现：作者说 "OOP is as much an experience as a technology"（OOP 一半是技术一半是经验）——因为运行时机制（属性搜索、方法绑定）只占一半，另一半是"如何组织代码"的设计智慧，只能靠项目历练。
- 设计原因：教学法考量——先给心智模型（mental model），再给语法细节。就像学开车先了解"方向盘转、车轮转"的整体感觉，再学换挡细节。
- 解决什么实际问题：防止初学者"只见树木不见森林"——术语（superclass、protocol methods.....）第一次看到会懵，但把它们都挂到"一棵树 + 一个特殊参数"的心智模型上，就全通了。
- 初学者误区：① 试图一次背下所有术语，焦虑内耗——

本章之后的全部内容都会反复用到这些词，自然就熟了；
② 以为“不会 OOP 术语就学不会”——术语是经验的浓缩，先理解机制，术语水到渠成。

26.4 属性继承搜索 (Attribute Inheritance Search)

英文原文

The good news is that OOP is much simpler to understand and use in Python than in some other languages like C++ or Java. As a dynamically typed scripting language, Python removes much of the syntactic clutter and complexity that clouds OOP in other tools. In fact, much of the OOP story in Python boils down to this expression: `object.attribute`. We've been using this expression throughout the book to access module attributes, call methods of objects, and so on. When we say this to an object that is derived from a class statement, however, the expression kicks off a search in Python—it searches a tree of linked objects, looking for the first appearance of attribute that it can find. When classes are involved, the preceding Python expression effectively translates to the following in natural language: Find the first occurrence of attribute by looking in object, then in all classes above it, from bottom to top and left to right. In other words, attribute fetches are simply tree searches. The term inheritance i

s applied to it because objects lower in a tree inherit attributes attached to objects higher in that tree. As the search proceeds from the bottom up, in a sense, the objects linked into a tree are the union of all the attributes defined in all their tree parents, all the way up the tree. In Python, this is all very literal: we really do build up trees of linked objects with code, and Python really does climb this tree at runtime searching for attributes every time we use the `object.attribute` expression. To make this more concrete, Figure 26-1 sketches an example of one of these trees. Figure 26-1. A class tree: instances (I1 and I2), a class (C1), and superclasses (C2 and C3) In this figure, there is a tree of five objects labeled with variables, all of which have attached attributes, ready to be searched. More specifically, this tree links together three class objects (the ovals C1, C2, and C3) and two instance objects (the rectangles I1 and I2) into an inheritance-search tree. Notice that in the Python object model, classes and the instances you generate from them are two distinct object types: Classes — Serve as instance factories. Their attributes provide behavior—data and functions—that is inherited by all the instances generated from them (e.g., a function to compute an employee's salary from pay and hours). Instances — Represent the concrete items in a program's domain. Their attributes record data that varies per specific object (e.g., an employee's pay rate and hours worked). In terms of s

search trees, an instance inherits attributes from its class, and a class inherits attributes from all classes above it in the tree. In Figure 26-1, we can further categorize the ovals by their relative positions in the tree. We usually call classes higher in the tree (like C2 and C3) superclasses; classes lower in the tree (like C1) are known as subclasses. These terms refer to both relative tree positions and roles. Superclasses provide behavior shared by all their subclasses, but because the search proceeds from the bottom up, subclasses may override behavior defined in their superclasses by redefining superclass names lower in the tree.¹ As these last few words are really the crux of the matter of software customization in OOP, let's expand on this concept. Suppose we build up the tree in Figure 26-1, and then say this: `I2.w` Right away, this code invokes inheritance. Because this is an `object.attribute` expression, it triggers a search of the tree in Figure 26-1—Python will search for the attribute `w` by looking in `I2` and above. Specifically, it will search the linked objects in this order: `I2`, `C1`, `C2`, `C3` and stop at the first attached `w` it finds (or raise an error if `w` isn't found at all). In this case, `w` won't be found until `C3` is searched because it appears only in that object. In other words, `I2.w` resolves to `C3.w` by virtue of the automatic search. In OOP terminology, `I2` "inherits" the attribute `w` from `C3`. Ultimately, the two instances inherit four attributes from their classes: `w`, `x`, `y`, and `z`. Other attribut

e references will wind up following different paths in the tree. For example: - I1.x and I2.x both find x in C1 and stop because C1 is lower than C2. - I1.y and I2.y both find y in C1 because that's the only place y appears. - I1.z and I2.z both find z in C2 because C2 is further to the left than C3. - I2.name finds name in I2 without climbing the tree at all. Trace these searches through the tree in Figure 26-1 to get a feel for how inheritance searches work in Python. The first item in the preceding list is perhaps the most important to notice—because C1 redefines the attribute x lower in the tree, it effectively replaces the version above it in C2. As you'll see in a moment, such redefinitions are at the heart of software customization in OOP—by redefining and replacing the attribute, C1 effectively customizes what it inherits from its superclasses.

中文翻译

好消息是：在 Python 中理解和运用 OOP 比在 C++ 或 Java 等其他语言中简单得多。作为一种动态类型脚本语言，Python 去除了其他工具中遮蔽 OOP 的大量语法冗余和复杂性。事实上，Python 的 OOP 故事很大程度归结为这样一个表达式：object.attribute（对象.属性）我们在整本书中一直在用这个表达式访问模块属性、调用方法等等。然而，当我们对一个由 class 语句派生出的对象使用这个表达式时，它会在 Python 中启动一次搜索（search）——它搜索一棵由链接对象构成的树，寻找它能找到的 attribute 的第一次出现。当涉及类时，前面的 Python 表达式实际上可以

翻译成如下自然语言：先在 object 中查找 attribute 的第一次出现，然后在它上方的所有类中查找，自底向上、从左到右。换句话说，属性获取 (attribute fetch) 就是树的搜索。之所以用继承 (inheritance) 这个词，是因为树中位置较低的对象继承位置较高对象上挂载的属性。随着搜索自下而上进行，在某种意义上，链接在树中的对象是所有树中祖先（一直追溯到树顶）上定义的所有属性的并集。在 Python 中，这一切都是非常字面化的 (literal)：我们确实用代码构建由链接对象组成的树，Python 也确实在每次使用 object.attribute 表达式时在运行时爬树搜索属性。为了更具体地说明，图 26-1 勾画了这样一棵树的一个示例。图 26-1. 一棵类树：实例 (I1 和 I2)、一个类 (C1)、以及超类 (C2 和 C3) 在这张图中，有一棵由五个带有变量标签的对象组成的树，它们都挂载了属性，随时可被搜索。更具体地说，这棵树将三个类对象 (class objects, 椭圆 C1、C2、C3) 和两个实例对象 (instance objects, 矩形 I1、I2) 链接成一个继承搜索树。请注意，在 Python 对象模型中，类以及你从类生成的实例是两种截然不同的对象类型：类 (Classes) ——充当实例工厂。它们的属性提供行为——数据和函数——被从它们生成的所有实例继承（例如，一个根据薪水 (pay) 和工时 (hours) 计算员工工资 (salary) 的函数）。实例 (Instances) ——代表程序领域中的具体条目。它们的属性记录因具体对象而异的属性数据（例如，员工的薪水标准和已工作时长）。就搜索树而言，实例从它的类继承属性，类从树中它上方的所有类继承属性。在图 26-1 中，我们可以根据椭圆在树中的相对位置进一步分类。我们通常把树中位置较高的类（如 C2 和 C3）称为超类 (superclasses)；树中位置较低的类（如 C1）称为

子类 (subclasses) 。这些术语既指相对树位置也指角色。超类提供其所有子类共享的行为，但由于搜索自下而上进行，子类可以通过在树中较低位置重新定义超类的名字来覆盖 (override) 超类中定义的行为。¹由于最后这几句话正是 OOP 中软件定制 (customization) 的要害所在，让我们展开讲讲这个概念。假设我们构建了图 26-1 中的树，然后写出这样一行代码：`I2.w` 这一行代码立刻调用了继承机制。因为这是一个 `object.attribute` 表达式，它会触发对图 26-1 中树的搜索——Python 会在 `I2` 及它上方查找属性 `w`。具体来说，它会按这个顺序搜索链接对象：`I2`, `C1`, `C2`, `C3` 并在找到第一个挂载的 `w` 时停止（如果完全找不到 `w`，则抛出错误）。在这个例子中，`w` 只出现在 `C3` 中，所以要搜到 `C3` 才会找到。换句话说，`I2.w` 凭借自动搜索解析为 `C3.w`。用 OOP 术语说，`I2` 从 `C3` “继承”了属性 `w`。最终，这两个实例从它们的类继承了四个属性：`w`、`x`、`y` 和 `z`。其他属性引用则会在树中走不同的路径。例如：- `I1.x` 和 `I2.x` 都在 `C1` 中找到 `x` 并停止，因为 `C1` 比 `C2` 位置更低。- `I1.y` 和 `I2.y` 都在 `C1` 中找到 `y`，因为 `y` 只出现在这一处。- `I1.z` 和 `I2.z` 都在 `C2` 中找到 `z`，因为 `C2` 比 `C3` 更靠左。- `I2.name` 在 `I2` 中就找到了 `name`，根本没有爬树。请顺着图 26-1 的树追踪这些搜索过程，体会 Python 中继承搜索是如何工作的。上面列表中的第一项也许是最值得注意的——因为 `C1` 在树中较低位置重新定义了属性 `x`，它实际上替换 (replaces) 了 `C2` 中位于它上方的那个版本。你马上就会看到，这种重新定义正是 OOP 中软件定制的核心——通过重新定义并替换属性，`C1` 实际上定制了它从超类继承来的东西。

深度理解

- 核心概念：属性获取 = 树搜索。规则一句话：先在实例里找，再沿着类继承链自底向上、从左到右找，找到第一个就停。I2.w 解析成 C3.w, I1.x 命中 C1.x 而不是 C2.x——位置越低、越靠左，优先级越高。
- 底层实现：CPython 中 `object.attribute` 最终会调用类型对象的 `tpgetattr`（对应 `getattr`）：先查实例自身的 `dict`，再沿 `mro`（方法解析顺序）逐个类查 `dict`。注意一个容易被忽略的细节：类对象也有 `dict`，实例继承的是类 `dict` 中的名字。图 26-1 中的“树”就是现代 Python 的 `mro` 的可视化（MRO 由 C3 线性化算法计算，第 28 章详述）。
- 设计原因：为什么“低处优先”？因为覆盖（`override`）是 OOP 定制的基石。父类定义通用行为，子类在更低位置重新定义同名属性，就自动“赢”了——不需要修改父类，不需要任何声明，纯靠搜索顺序天然实现。这就是“编程靠定制而非修改”的运行时根基。
- 解决什么实际问题：团队协作时，通用逻辑放超类写一次，各子类按需覆盖；新需求来了，只写新子类，旧代码零改动。搜索顺序保证“离你最近的定义说了算”，和日常生活中“本地规定优先于全国规定”同理：城市限行规则（子类）可以覆盖国家通用规则（超类），但国家规则自动适用于所有城市。
- 初学者误区：① 把 `object.attribute` 想成“直接读字典”——它是一次搜索过程，结果取决于对象在树中的位置，

同一个名字在不同对象上可能解析到不同定义；② 以为覆盖（override，同名替换）和重载（overload，同名字不同参数）是一回事——Python 没有 C++/Java 那种参数式重载，后者第 29 章会澄清；③ 以为“下面的覆盖会让上面的属性消失”——不，C2.x 还在原处，只是搜索优先选中 C1.x；想要 C2.x 依然可以直接写 C2.x 显式取用；④ 忘记报错规则：全树搜遍都没有，抛 `AttributeError`。

26.5 类与实例 (Classes and Instances)

英文原文

Although they are technically two separate object types in the Python model, the classes and instances we put in these trees are almost identical—each type's main purpose is to serve as another kind of namespace—a package of variables, and a place where we can attach attributes. If classes and instances therefore sound like modules, they should; however, the objects in class trees also have automatically searched links to other namespace objects, and classes and instances correspond to statements and calls, respectively, not entire files. The primary difference between classes and instances is that classes are a kind of factory for generating instances. For example, in a realistic application, we might have an `Employee` class that defines what it means to be an employee; from that class, we generate actual `Employee` instances. This is another d

ifference between classes and modules—we only ever have one instance of a given module in memory (that's why we have to reload a module to get its new code), but with classes, we can make as many unique instances as we need. Operationally, classes will usually have functions attached to them (e.g., `computeSalary`), and the instances will have more basic data items used by the class's functions (e.g., `hoursWorked`). In fact, the object-oriented model is not that different from the classic data-processing model of programs plus records—in OOP, instances are like records with "data," and classes are the "programs" for processing those records. In OOP, though, we also have the notion of an inheritance hierarchy, which supports software customization better than earlier models.

中文翻译

虽然在 Python 模型中，类和实例技术上算是两种不同的对象类型，但放进这些树中的类和实例几乎是一样的——每种类型的主要目的都是充当另一种命名空间（namespace）——一个变量的包裹体，一个我们可以挂载属性的地方。如果类和实例因此听起来很像模块，那也情有可原；不过，类树中的对象还具有自动被搜索的、指向其他命名空间对象的链接，而且类和实例分别对应于语句（statement）和调用（call），而不是整个文件。类和实例的主要区别在于：类是生成实例的一种工厂（factory）。例如，在一个真实的应用程序中，我们可能有一个 `Employee`（员工）类，定义“当一名员工意味着什么”；从这个类中，我们生成实际

的 Employee 实例。这是类与模块的另一个区别——一个给定的模块在内存中永远只有一个实例（这就是为什么我们必须 reload 模块才能拿到它的新代码），而用类，我们可以按需制造任意多个独一无二的实例。在操作层面，类通常挂载函数（例如 computeSalary，计算工资），实例则挂载被类的函数使用的基础数据项（例如 hoursWorked，已工作时长）。事实上，面向对象模型与经典的"程序 + 记录"（programs plus records）数据处理模型并没有太大不同——在 OOP 中，实例就像带"数据"的记录，类则是处理这些记录的"程序"。不过，在 OOP 中我们还有继承层次的概念，它比早期模型更好地支持软件定制。

深度理解

·核心概念：类 = 实例工厂；类与实例都是命名空间。蓝图（类）与具体产品（实例）是 OOP 最重要的一对关系。类负责"共享的行为定义"，实例负责"各异的属性数据"。

·底层实现：类和实例的 dict 都是普通字典。实例的 dict 装自己的数据（如 name），类的 dict 装函数和类级数据。类对象的内存结构里多了 bases（指向超类的元组）；实例则有一个指向类的 class 指针——这正是属性搜索往上爬的入口。

·设计原因：为什么叫"工厂"而不是"模板"？因为调用类每次都会生产新对象——正如蛋糕模具可以反复倒出无数块蛋糕。对比模块：模块加载一次就是一个对象，要新代码只能 reload；类的实例却可以无限生成。作者还给出经典类比：实例是"记录"，类是处理记录的"程序"——OOP

就是把数据和处理数据的方法绑在一起的旧模型升级版。

·解决什么实际问题：一个公司系统里"员工"是一类概念，但每个员工的数据都不同。类定义规则（怎么算工资），实例存放数据（张三的工时）。想加一百个员工？调用类一百次即可，天然支持。

·初学者误区：① 把"类"当成"对象集合"或"数据结构定义"——类首先是可调用的工厂；② 以为实例会自动复制类里的所有属性——实例不复制任何东西，只是链接到类，属性查找时向上借；③ 认为类里每个方法都必须操作实例——类属性（类级数据）也是合法的，只是作用域不同（本章 26.7 会展开）；④ 误以为"用类 = 必须重写所有代码"——恰恰相反，类是"复用已有代码"的工具。

26.6 方法调用 (Method Calls)

英文原文

In the prior section, we saw how the attribute reference `I2.w` in our example class tree was translated to `C3.w` by the inheritance search procedure in Python. Perhaps just as important to understand as the inheritance of attributes, though, is what happens when we try to call methods—functions attached to classes as attributes. If this `I2.w` reference is a function call, what it really means is "call the `C3.w` function to process `I2`." That is, Python will automatically map the call `I2.w()` into the call `C3.w(I2)`, passing in the instance as the fi

rst argument to the inherited function as the implied subject of the call. In fact, whenever we call a function attached to a class in this fashion, an instance of the class is always implied. This implied subject is part of the reason we refer to this as an object-oriented model—there is always a subject object when an operation is run. In a more realistic example, we might invoke a method called `giveRaise` attached as an attribute to an `Employee` class; such a call has no meaning unless qualified with the employee to whom the raise should be given. As you'll see in more detail later, Python passes in the implied instance to a special first argument in the method, called `self` by strong convention. Methods go through this argument to process the subject of the call. As you'll also learn later, methods can be called either through an instance—`pat.giveRaise()`—or through a class—`Employee.giveRaise(pat)`—and both forms serve purposes in our scripts. In fact, these calls illustrate both of the key ideas in OOP; to run a `pat.giveRaise()` method call, Python:

1. First looks up `giveRaise` from `pat`, by inheritance.
2. Then passes `pat` to the located `giveRaise` function, in the special `self` function argument.

When you run the call `Employee.giveRaise(pat)`, you're just performing both steps yourself. This description is technically just the default case (Python has additional method types you'll meet later, called static and class methods), but it applies to the vast majority of the OOP code written in

the language. To see how methods receive their subjects, though, we need to move on to some code.

中文翻译

在前一节中，我们看到了示例类树中的属性引用 `I2.w` 如何被 Python 的继承搜索过程翻译成 `C3.w`。与属性继承同样重要的，或许是我们试图调用方法（methods）——作为属性挂载在类上的函数——时会发生什么。如果这个 `I2.w` 引用是函数调用，它真正的意思是“调用 `C3.w` 函数来处理 `I2`”。也就是说，Python 会自动把调用 `I2.w()` 映射成调用 `C3.w(I2)`，把实例作为第一个参数传给继承来的函数，作为调用的隐含主体（subject）。事实上，每当我们以这种方式调用挂载在类上的函数时，总会隐含一个该类的实例。这个隐含的主体正是我们称之为面向对象（object-oriented）模型的部分原因——每次运行一个操作时，总有一个主体对象。在一个更真实的例子中，我们可能调用一个叫 `giveRaise`（加薪）的方法，它作为属性挂在 `Employee` 类上；如果不限定是给哪个员工加薪，这样的调用毫无意义。稍后你会更详细地看到，Python 把这个隐含实例传给方法中一个特殊的第一个参数，按强烈的惯例它被称为 `self`。方法通过这个参数来处理调用的主体。你还会学到，方法既可以通过实例调用——`pat.giveRaise()`——也可以通过类调用——`Employee.giveRaise(pat)`——两种形式在脚本中都各有用途。事实上，这两种调用展示了 OOP 的两个关键思想；要执行 `pat.giveRaise()` 这个方法调用，Python：1. 先通过继承从 `pat` 查找 `giveRaise`。2. 然后把 `pat` 传给定位到的 `giveRaise` 函数，放在特殊的 `self` 函数参数中。当你执行 `Employee.giveRaise(pat)` 这个调用时，你只是手

动完成了这两步而已。严格来说，上述描述只是默认情形（Python 还有额外的方法类型，稍后你会遇到，称为静态方法（static method）和类方法（class method）），但它适用于该语言中绝大多数 OOP 代码。不过，要看到方法如何接收它们的主体，我们需要进入一些代码。

代码分析

下面是这一机制的最小复现。类对象与实例对象的行为在解释器中逐条执行：

```
class C2: ...          #
class C3:
    w = 10              # w
class C1(C2, C3):      # C1 C2 C3
    pass

I2 = C1()              #
print(I2.w)            # I2 -> C1 -> C2 -> C3 w=10

class Employee:        #
    def giveRaise(self, percent): # self
        self.pay *= (1.0 + percent) # self ""

pat = Employee()        #
pat.pay = 1000.0        #
pat.giveRaise(0.1)      # Employee.giveRaise(pa...
print(pat.pay)          # 1100.0
```

解释器如何处理：

1. 当解释器执行 `I2.w` 时，`LOADATTR` 指令触发属性查找：先查 `I2.dict`（没有），再沿 `I2.class` 找到 `C1`，查 `C1.dict`（没有），再到 `C1.bases` 中的 `C2`（没有）、`C3`（找到 `w`）。于是 `I2.w` 就是 `C3.w` 绑定到实例上的结果。

2. 当 `w` 是函数（如 `giveRaise`）且以 `pat.giveRaise(...)` 形式调用时，关键魔法在 `LOADMETHOD` 指令：Python 从实例找到函数对象后，不直接调用它，而是把 `pat` 作为第一个位置参数打包进去，生成一个“绑定方法”（bound method）对象，再与你显式传入的实参合并，最终调用的是 `giveRaise(pat, 0.1)`。这正是书中两步（查找 + 传 `self`）的字节码级体现。

3. 如果你偷懒用 `Employee.giveRaise(pat)`——从类直接取函数，没有实例可绑定，Python 就只拿到原始函数，两个参数得自己传：`Employee.giveRaise(pat, 0.1)`。若只写 `Employee.giveRaise(0.1)`，`0.1` 会被当作 `self`，随后 `self.pay` 必然 `AttributeError`。这是初学者最常见的报错之一。

设计思想：把“主体”显式放在第一个参数，而不是像 C++/Java 那样用隐藏的 `this` 指针，是 Python 的刻意设计——`self` 在方法头和函数体中都显式出现，让属性访问的来源一目了然，符合“显式优于隐式”（explicit is better than implicit）的哲学（详见本章脚注 2）。代价是写起来多敲几个字母，收益是阅读和调试时零歧义。

深度理解

·核心概念：方法调用 = 两步：第一步按继承规则查找函数，第二步把实例塞进第一个参数。`l2.w()` 字面上等价于 `C3.w(l2)`。写 `Employee.giveRaise(pat)` 就是手动做这两步。

·底层实现：见上方代码分析——`LOADATTR/LOADMET`

HOD 指令 + 绑定方法对象。绑定方法对象里存着（函数，实例）两个引用，调用时才合并参数，所以每次 `pat.giveRaise` 取出的绑定方法都是新的临时对象（这就是 `id(pat.giveRaise) != id(pat.giveRaise)` 的原因）。

·设计原因：为什么任何方法都“隐含一个主体”？因为类挂载的函数是为“处理某类具体对象”而生的——`giveRaise` 不给具体员工就没意义（给谁加薪？加多少？）。“总有一个被处理的对象”正是“面向对象”四个字的字面含义。

·解决什么实际问题：同一个类函数 `computeSalary` 被一百个实例调用，每次自动带上“给哪个实例算工资”的上下文，调用方无需手动传参，代码既简洁又不易传错对象。

·初学者误区：① 定义方法时漏写 `self`——调用时得到 `TypeError: ... takes 1 positional argument but 2 were given`（函数只要 1 个参数，实例却自动补了一个）；② 以为 `self` 是关键字——它只是强惯例的名字，写成 `this` 或 `me` 也能运行，但绝不要这么干，社区代码默认就是 `self`；③ 以为“通过实例调用”和“通过类调用”有本质区别——两者最终执行的函数完全相同，只是前者自动传实例、后者手动传。

26.7 编写类树 (Coding Class Trees)

英文原文

Although we are speaking in the abstract here, there is tangible code behind all these ideas, of course. We construct trees and their objects with class statement

s and class calls, which we'll explore in more detail later. In short, though:

- Each class statement generates a new class object.
- Each time a class is called, it generates a new instance object.
- Instances are automatically linked to the classes from which they are created.
- Classes are automatically linked to their superclasses according to the way we list them in parentheses in a class header line; the left-to-right order there gives the order in the tree. To build the tree in Figure 26-1, for example, we would run Python code of the sort in Example 26-1. Like function definition, classes are normally coded in module files and are run during an import (the guts of the following class statements are omitted here for brevity, though ... qualifies as a no-op statement per Chapter 13 if run live). Example 26-1. classtree1.py

```
python class C2: ... # Make class objects (ovals)
class C3: ...
class C1(C2, C3): ... # Linked to superclasses - in
this order
I1 = C1() # Make instance objects (rectangles)
I2 = C1() # Linked to their class
Here, we build the three class objects by running three class statements,
and make the two instance objects by calling the class C1 twice—as though
it were a function (Python lumps classes and functions together as "callable"
objects invoked with parentheses, though def requires header parentheses and
class does not). The instances remember the class they were made from, and
the class C1 remembers its listed superclasses. Technically, th
```

is example uses something called multiple inheritance, which simply means that a class has more than one superclass above it in the class tree—a useful technique when you wish to combine multiple tools. In Python, if there is more than one superclass listed in parentheses in a class statement (like C1's here), their left-to-right order gives the order in which those superclasses will be searched for attributes by inheritance. The leftmost version of a name is used by default, though you can always choose a name by asking for it from the class it lives in (e.g., C3.z). The search also picks names to the right over higher duplicates, but we can safely ignore that for now. Because of the way inheritance searches proceed, the object to which you attach an attribute turns out to be crucial—it determines the name's scope. Attributes attached to instances pertain only to those single instances, but attributes attached to classes are shared by all their subclasses and instances. Later, we'll study the code that hangs attributes on these objects in depth. As you'll find, it's all about where an assignment is run:

- Attributes are usually attached to classes by assignments made at the top level in class statement blocks, and not nested inside function def statements there.
- Attributes are usually attached to instances by assignments to the special argument passed to functions coded inside classes, called `self`. For example, classes provide behavior for their instances with functions we create by

coding `def` statements inside class statements. Because such nested `def` statements assign function names within the class, they wind up attaching attributes to the class object that will be inherited by all instances and subclasses—as is Example 26-2, which lists changed parts in bold. Example 26-2. `classtree2.py`

```
python class C2: ... # Make superclass objects class C
3: ... class C1(C2, C3): # Make and link class C1 def set
name(self, who): # Assign name: C1.setname self.nam
e = who # Self is either I1 or I2 I1 = C1() # Make two
instances I2 = C1() I1.setname('sue') # Sets I1.name t
o'sue'I2.setname('bob') # Sets I2.name to'bob'
```

`print(I1.name)` # Prints'sue' There's nothing syntactically unique about `def` in this context. Operationally, though, when a `def` appears inside a class like this, it is usually known as a method, and it automatically receives a special first argument—called `self` by very strong convention—that provides a handle back to the instance to be processed. Any values you pass to the method yourself go to arguments after `self` (here, to `who`).² Because classes are factories for multiple instances, their methods usually go through this automatically passed-in `self` argument whenever they need to fetch or set attributes of the particular instance being processed by a method call. In the preceding code, `self` is used to store a name in one of two instances. Like simple variables, attributes of classes and instances are not declared ahead of time, but spring into exis

tence the first time they are assigned values. When a method assigns to a self attribute, it creates or changes an attribute in an instance at the bottom of the class tree (i.e., one of the rectangles in Figure 26-1) because self automatically refers to the instance being processed—the subject of the call. In fact, because all the objects in class trees are just namespace objects, we can fetch or set any of their attributes by going through the appropriate names. Saying C1.setname is as valid as saying I1.setname, as long as the names C1 and I1 are in your code's scopes.

中文翻译

虽然我们在这里谈得很抽象，但所有这些思想背后当然有实实在在的代码。我们用 class 语句和类调用来构建树及其对象，后面我们会更详细地探讨。简而言之：- 每个 class 语句都会生成一个新的类对象。- 每次调用一个类，都会生成一个新的实例对象。- 实例自动链接到创建它们的类。- 类自动链接到它们的超类，链接方式取决于我们在 class 头行括号中列出的超类顺序；那里从左到右的顺序就给出了树中的搜索顺序。例如，要构建图 26-1 中的树，我们会运行类似示例 26-1 的 Python 代码。和函数定义一样，类通常编写在模块文件中，并在导入 (import) 时运行（为简洁起见，下面 class 语句的“内脏”被省略了，不过如果实际运行，... 按第 13 章的说法是一种空操作 (no-op) 语句）。示例 26-1. classtree1.py

```
python class C2: ... # 生成类对象 (椭圆)
class C3: ...
class C1(C2, C3): ... # 链接到超类——按照这个顺序
I1 = C1() # 生成实例对象 (矩形)
I2 = C1
```


() # 链接到它们的类在这里，我们通过运行三条 class 语句构建了三个类对象，并通过两次调用类 C1 制造了两个实例对象——把 C1 当作函数来调用（Python 把类和函数归为一类“可调用”（callable）对象，都用括号调用，只是 def 需要头行括号而 class 不需要）。实例记住它们由哪个类创建，类 C1 记住它列出的超类。严格来说，这个示例使用了所谓的多重继承（multiple inheritance），意思就是类树中一个类的上方有多个超类——当你想组合多个工具时，这是一个有用的技术。在 Python 中，如果 class 语句的括号里列了多个超类（就像这里的 C1），它们从左到右的顺序就决定了继承搜索这些超类属性的顺序。默认情况下采用最左边的名字版本，不过你总是可以指定从它所在的类中去取某个名字（例如 C3.z）。搜索还会优先选择右侧的名字而不是更高的重复项，但我们现在可以放心地忽略这一点。由于继承搜索的运行方式，你把属性挂载在哪个对象上就变得至关重要——它决定了该名字的作用域（scope）。挂载在实例上的属性只属于那一个实例，而挂载在类上的属性会被它的所有子类 and 实例共享。稍后我们将深入研究把属性挂到这些对象上的代码。你会发现，这一切都取决于赋值（assignment）在哪里运行：- 属性通常通过在 class 语句块的顶层进行赋值而挂到类上，而不是嵌在里面的函数 def 语句中。- 属性通常通过给类内函数接收的特殊参数（称为 self）赋值而挂到实例上。例如，类通过我们在 class 语句内编写 def 语句创建的函数来为实例提供行为。因为这种嵌套的 def 语句在类内赋值函数名，它们最终会把属性挂到类对象上，从而被所有实例和子类继承——正如示例 26-2 所示（变更部分以粗体列出）。示例 26-2. classtree2.py

```
python class C2: ... # 生成超类对象class C3: ... class C
```

```
1(C2, C3): # 生成并链接类 C1
def setname(self, who): # 赋值 name: C1.setname
self.name = who # self 是 I1 或 I2
I1 = C1() # 生成两个实例
I2 = C1()
I1.setname('sue') # 将 I1.name 设为'sue'
I2.setname('bob') # 将 I2.name 设为'bob'
print(I1.name) # 打印'sue'在这个上下文中, def 在语法上没有任何特殊之处。但在操作层面, 当 def 像这样出现在 class 内时, 它通常被称为方法 (method), 并自动接收一个特殊的第一个参数——按非常强的惯例称为 self——它提供了回到被处理实例的"手柄"。你自己传给方法的任何值都放在 self 之后的参数中 (这里就是 who)。2因为类是多个实例的工厂, 方法在需要通过方法调用处理某个特定实例的属性时, 通常都要经过这个自动传入的 self 参数去读取或设置属性。在上面的代码中, self 被用来把名字存进两个实例中的一个。和简单变量一样, 类和实例的属性 (attributes) 不需要预先声明, 而是在第一次被赋值时才"凭空出现" (spring into existence)。当方法给 self 的属性赋值时, 它会在类树底部的实例中 (即图 26-1 中的某个矩形) 创建或修改一个属性, 因为 self 自动指向被处理的实例——即调用的主体。事实上, 因为类树中的所有对象都只是命名空间对象, 我们可以通过适当的名字获取或设置它们的任何属性。只要 C1 和 I1 在你的代码作用域中, 写 C1.setname 和写 I1.setname 一样合法。
```

代码分析

示例 26-1 (classtree1.py) ——搭建树骨架

```

class C2: ...                                #
class C3: ...
class C1(C2, C3): ...                        # —
I1 = C1()                                    #
I2 = C1()                                    #

```

解释器如何处理：

1. 每一条 class 语句执行时，Python 都会创建一个新的类对象（一个 type 类型的实例），并把名字 C2 绑定到它身上。类体（这里是 ...）里的代码在语句执行时立即运行一次，用来填充类对象的 dict。
2. class C1(C2, C3) 执行时，Python 先创建 C1 的类对象，然后把 C2、C3 记录在 C1.bases 中——这就是“树”的物理载体。搜索时解释器从 bases 取出超类列表，按序向上查找。
3. I1 = C1() 把类当函数调用：CPython 找到 C1 的元类（type），调用其 tpcall，分配一块实例内存，并把实例的 class 指针设为 C1。两次调用产生两个完全独立的实例对象，各自有独立的 dict。
4. 至此，I1.dict、I2.dict 都为空，但 I1.任何名字的搜索会沿 I1 → C1 → C2 → C3 这条链进行——树已经可以爬了。

设计思想：树的构建语法极其廉价——一个 class 语句、一次带括号的调用，没有任何“注册”“声明”仪式。这是 Python 动态性的体现：类型信息不是编译期决定的，而是运行期由语句执行“长”出来的。

示例 26-2 (classtree2.py) ——用方法给实例挂数据

```

class C2: ...                                #
class C3: ...
class C1(C2, C3):                            # C1
    def setname(self, who):                  # nameC1.setname
        self.name = who                     # self I1 I2
I1 = C1()                                    #
I2 = C1()
I1.setname('sue')                            # I1.name 'sue'
I2.setname('bob')                            # I2.name 'bob'
print(I1.name)                               # 'sue'

```

解释器如何处理：

1. `class C1(...)` 的类体执行时，其中的 `def setname(...)` 按普通 `def` 规则创建一个函数对象，并把名字 `setname` 写进 `C1.dict`。注意：此刻 `self.name = who` 一行代码都没执行——它只是函数体的字节码，留到调用时才运行。
2. `I1.setname('sue')`：属性查找先看 `I1.dict`（没有 `setname`），上到 `C1.dict` 找到函数；解释器发现函数名在类字典中，于是生成绑定方法（bound method），把 `I1` 当作第一个实参，实际执行 `C1.dict['setname']`——即 `self` 收到 `I1`，`who` 收到 `'sue'`。
3. 函数体执行 `self.name = who`：这是对 `self` 对象（即 `I1`）的实例属性赋值。由于 `I1` 的 `dict` 里没有 `name`，解释器直接新建这个键——属性无需预声明，首次赋值即创建。同理，`I2.setname('bob')` 在 `I2` 自己的 `dict` 里创建 `name`，与 `I1` 互不干扰。
4. `print(I1.name)`：读属性时，在 `I1.dict` 中直接命中 `'sue'`，根本不需要爬树——这印证了前面“实例属性只属于单个实例”的作用域规则。

设计思想：注意分工——setname（行为）挂在类上，name（数据）挂在实例上。函数只此一份，被所有实例共享（省内存）；数据各有一份（保隔离）。这正是"类 = 程序，实例 = 记录"模型的第一份代码落地。

深度理解

·核心概念：属性挂在哪，作用域就多大。挂在实例上 → 只属于该实例；挂在类上 → 所有子类 and 实例共享。而"挂在哪"完全由赋值语句的执行位置决定：类体顶层赋值 → 类属性；方法里 self.xxx = ... → 实例属性。这是动态语言"属性凭空出现"（无声明）的特性。

·底层实现：赋值语句没有搜索过程——self.name = who 永远直接写入 self.dict，不会沿着继承链找已有属性去改。这一点和"读取时搜索"形成鲜明对比：读走链、写只在本地。类体顶层的 x = 1 则写进类对象的 dict，所有实例读取时都能在类上"借到"。

·设计原因：为什么读取要搜索而赋值不搜索？如果赋值也沿链搜索，一个 self.name = ... 可能把父类属性改掉，波及所有其他实例——灾难。设计成"写只在本地"，既保证每个实例数据隔离，也让 name 这种属性"第一次赋值才诞生"（省去预声明的负担）。

·解决什么实际问题：多实例场景下，方法只需写一份，通过 self 精准操作"当下这个"实例；不同实例的数据天然隔离，不会互相污染。这是"一个类、千万个实例"的工程基础。

·初学者误区：① 以为 `setname` 这种方法是"给类用的"——其实它是给实例用的工具，`self` 就是"当前正在被处理的那个实例"的代言人；② 混淆 `self` 与 `C1`——两者都能取到同一个函数，但只有 `self` 形式会自动带实例；③ 忘记多重继承的搜索顺序规则：从左到右（`C1(C2, C3)` 先查 `C2` 系再查 `C3` 系），最左边优先；④ 以为"类属性被子类覆盖后父类也变"——不会，覆盖只是在树中更低处挂了一个新名字，父类原名原样保留。

26.8 运算符重载 (Operator Overloading)

英文原文

As currently coded, our `C1` class doesn't attach a name attribute to an instance until the `setname` method is called. Indeed, referencing `I1.name` before calling `I1.setname` would produce an undefined name error. If a class wants to guarantee that an attribute like name is always set in its instances, it more typically will fill out the attribute at construction time, as demoed by Example 26-3.

```
Example 26-3. classtree3.py
python
class C2: ... # Make superclass objects
class C3: ...
class C1(C2, C3):
    def init(self, who):
        # Set name when constructed
        self.name = who
        # Self is either I1 or I2
        I1 = C1('sue') # Sets I1.name to 'sue'
        I2 = C1('bob') # Sets I2.name to 'bob'
        print(I1.name) # Prints 'sue'
If it's either coded or inherited, Python automatically calls a method named init each time an instance is generated from
```

m a class. The new instance is passed in to the self argument of `__init__` as usual, and any values listed in parentheses in the class call go to arguments two and beyond. The effect here is to initialize instances when they are made, without requiring extra method calls. The `__init__` method is known as the constructor because of when it is run. It's the most commonly used representative of a larger category called operator-overloading methods, which we'll explore in later chapters. Such methods are inherited in class trees as usual and have double underscores at the start and end of their names to make them distinct. Python runs them automatically when instances that support them appear in the corresponding operations, and they are mostly an alternative to using simple method calls. They're also optional: if omitted, the operations are not supported. If no `__init__` is present, class calls return an empty instance, without initializing it. For example, a custom set intersection might be coded as a method named `__intersect` called explicitly, or as a method named `__and__` that is called automatically by the `&` expression operator. Because the operator scheme makes instances look and feel more like built-in types, it allows some classes to provide a consistent and natural interface, and be compatible with code that expects a built-in type. Still, apart from the `__init__` constructor—which appears in most realistic classes—many programs may be better off with simpler named methods unless their o

jects are similar to built-ins. A giveRaise may make sense for an Employee, but an & might not.

中文翻译

按目前的写法，我们的 C1 类直到 setname 方法被调用之前，都不会在实例上挂载 name 属性。确实，在调用 l1.setname 之前就引用 l1.name 会产生未定义名字的错误。如果一个类想保证 name 这样的属性在其实例中总是被设置，它通常会在构造时（construction time）就把属性填好，如示例 26-3 所示。示例 26-3. classtree3.py

```
python class C2: ... # 生成超类对象class C3: ... class C1(C2, C3): def init(self, who): # 构造时设置 name self.name = who # self 是 l1 或 l2 l1 = C1('sue') # 将 l1.name 设为'sue' l2 = C1('bob') # 将 l2.name 设为'bob'print(l1.name) # 打印'sue'无论 init 是被定义还是被继承，Python 每次从类生成实例时都会自动调用这个名为 init 的方法。新实例照常被传给 init 的 self 参数，类调用括号中列出的任何值则进入第二个及之后的参数。这样做的效果是：实例被创建时就完成初始化，无需额外的方法调用。init 方法因其运行时机而被称为构造器（constructor）。它是一个更大的类别——运算符重载方法（operator-overloading methods）——中最常用的代表，我们将在后面的章节探讨这一类方法。这些方法照常类中被继承，名字的首尾都有双下划线以示区别。当支持它们的实例出现在相应运算中时，Python 会自动运行它们；它们大多是简单方法调用的替代方案。它们也是可选的：如果省略，相应的运算就不被支持。如果没有 init，类调用会返回一个空的实例，不做任何初始化。例如，一个自定义集合交集可以写成名为 intersect 的
```


方法显式调用，也可以写成名为 `and` 的方法由 `&` 表达式运算符自动调用。因为运算符方案让实例看起来、用起来更像内置类型，它允许某些类提供一致而自然的接口，并与期待内置类型的代码兼容。不过，除了出现在大多数真实类中的 `init` 构造器之外，许多程序也许更适合使用更简单的命名方法——除非它们的对象与内置类型相似。对 `Employee` 来说，`giveRaise` 说得通，但 `&` 也许并不合适。

代码分析

示例 26-3 (`classtree3.py`) ——用 `__init__` 保证属性初始化

```
class C2: ...                                #
class C3: ...
class C1(C2, C3):
    def __init__(self, who):                 # name
        self.name = who                     # self I1 I2
I1 = C1('sue')                              # I1.name 'sue'
I2 = C1('bob')                              # I2.name 'bob'
print(I1.name)                              # 'sue'
```

解释器如何处理：

1. 执行 `I1 = C1('sue')` 时，CPython 的调用流程是：分配实例内存（new，默认从 `object` 继承）→ 查找 `init`（沿继承链！如果 `C1` 自己没有定义，会继续往 `C2`、`C3`、`object` 里找）→ 以 `(self, 'sue')` 为参数调用它。这里 `init` 在 `C1.dict` 中直接命中，于是 `self.name = 'sue'` 执行，实例一出生就带上了 `name`。
2. 注意 `init` 是普通方法，一样遵守“两步调用”：实例自动作为 `self`，括号里的 `'sue'` 落到第二个参数 `who`。所以 `C1`

('sue') 与"先造空实例再手动调 C1.init(新实例,'sue')"在语义上等效，只是前者自动完成。

3. 若某个类没有 init 且继承链上也找不到，类调用就只执行默认的内存分配，返回一个 dict 为空的"空白实例"——这就是"没有构造器时实例以空命名空间开始生命"的机制。

4. 双下划线名字 (dunder) 在类 dict 里并无特殊存储地位——它就是普通字符串键，特殊之处只在于解释器约定：特定运算发生时自动查找对应名字（+ 找 add、& 找 and、print 找 str.....）。自动调用的触发点由解释器的指令级逻辑硬编码，而不是类自身声明"我要响应 &"。

设计思想：init 把"初始化"从使用者的义务（记得调 setname）变成类的承诺（一出生就完整）。这消除了"忘记初始化"这类最隐蔽的 bug 源头。同时它也示范了运算符重载的通用模式：名字带双下划线 = 与解释器约定的协议（protocol），类通过实现协议方法来融入语言本身的语法生态。

深度理解

·核心概念：构造器 init 是自动调用的初始化钩子，是"运算符重载方法"家族（add、str、getitem.....）中最常用的一个。特性：双下划线命名、自动调用、沿继承链查找、可选（省略则该运算不支持）。

·底层实现：init 其实不是"构造函数"——真正的内存分配由 new 完成，init 只负责"填充内容"。C++/Java 的构造函数两者合一，Python 拆成两步，这是初学者容易误解

的点。运算符重载的查找走的是"特殊查找协议" (add 等)，比普通属性查找规则更多 (例如还会检查右操作数类型、radd 回退等)，第 29 章详述。

·设计原因：为什么让运算自动触发而非手动调用？为了让自定义对象无缝融入语言语法——`obj & other`、`len(obj)`、`print(obj)` 都能按类定制的语义工作，调用方无需知道对象的内部接口，这与内置类型的行为方式完全一致。

·解决什么实际问题：示例 26-2 里 `name` 是"用到才出现"的，忘调 `setname` 就 `AttributeError`——`__init__` 把"必然存在"变成现实，消灭一类运行时错误。运算符重载则让第三方类 (如 NumPy 数组) 支持 `a + b`、`a[i]` 等原生语法。

·初学者误区：① 以为 `__init__` 是"分配内存"的构造函数——它只是初始化器，分配是 `new` 干的；② 以为所有双下划线名字都是魔法——`__init__`、`__str__` 等特定名字才有协议意义，乱造双下划线名字没有特殊效果；③ 滥用运算符重载——作者明确提醒：对象不像内置类型就别硬上 `&`、`+`，语义混乱的运算符是坑，普通命名方法 (`intersect`) 往往更清晰；④ 忘了"可选性"——没写 `__init__` 不会报错，实例只是空着，所以"写了才生效、忘写静默变空"反而要警惕。

26.9 OOP 的本质是代码复用 (OOP Is About Code Reuse)

英文原文

And that, along with a few syntax details, is most of the OOP story in Python. Of course, there's a bit more to it than just inheritance. For example, operator overloading is much more general than described so far—classes may also provide their own implementations of operations such as indexing, fetching attributes, printing, and more. By and large, though, OOP is about looking up attributes in trees with a special first argument in functions. So why would we be interested in building and searching trees of objects? Although it takes some experience to see how, when used well, classes support code reuse in ways that other Python program components cannot. In fact, this is, by most accounts, their highest purpose. With classes, we code by customizing existing software, instead of either changing existing code in place or starting from scratch for each new project. This turns out to be a powerful paradigm in realistic programming. At a fundamental level, classes are really just packages of functions and other names, much like modules. However, the automatic attribute inheritance search that we get with classes supports customization of software above and beyond what we can do with modules and functions. Moreover, classes provide a natural structure for code that packages and localizes both logic and names, and so aids in debugging. To be fair, because methods are simply functions with a special first argument, we can mimic some of their behavior by manually p

assing subject objects to simple functions. The participation of methods in class inheritance, though, allows us to naturally extend and customize software by coding subclasses with new methods, rather than modifying code that already works. There is really no such concept with modules and functions.

中文翻译

就这样，再加上一些语法细节，就是 Python 中 OOP 故事的大部分了。当然，它不止继承这么一点。例如，运算符重载比目前描述的通用得多——类还可以提供索引、属性获取、打印等运算的自定义实现。但总的说来，OOP 就是在树中查找属性，加上函数中的一个特殊第一参数。那么，我们为什么会对构建和搜索对象树感兴趣呢？虽然要看出这一点需要一些经验，但用得好的时候，类能以其他 Python 程序组件无法做到的方式支持代码复用（reuse）。事实上，在大多数人看来，这就是它们的最高使命。用类编程，我们是通过定制现有软件来写代码，而不是就地修改现有代码，也不是每个新项目都从零开始。这在现实编程中被证明是一种强大的范式。从根本上说，类其实只是一些函数和其他名字的包裹体，很像模块。然而，类带来的自动属性继承搜索，能支持超越模块和函数所能做到范围的软件定制。此外，类为代码提供了自然的结构（structure），把逻辑和名字打包并局部化，从而有助于调试。公平地说，因为方法只是带特殊第一参数的函数，我们可以通过手动把主体对象传给普通函数来模仿它们的部分行为。但方法参与类继承这一点，让我们可以通过编写带新方法的子类来自然地扩展和定制软件，而不是修改已经能工作的代码。模块和函数真的没有这样

的概念。

深度理解

·核心概念：本章最终答案——OOP 的最高目的是代码复用（code reuse）：通过"定制现有代码"写新程序，而不是"改现有代码"或"从零重写"。类 = 树搜索 + 特殊第一参数，就这两件武器。

·底层实现：复用不是拷贝——子类对象通过指针链接到超类，运行时搜索时"借用"父类的函数对象，函数字节码在内存中只有一份，被整棵树的实例共享。这正是"结构性复用"（复用定义而非复制代码）的物理体现。

·设计原因：为什么"定制"优于"修改"？修改已工作的代码会引入回归（regression）风险；继承则让新代码"长"在旧代码旁边，旧代码一行不动。生活例子：往词典里加一个新词条（子类），而不是改写已有词条（修改父类）——词典原有内容零风险。

·解决什么实际问题：新需求 → 写子类覆盖少量方法；跨项目复用 → 直接继承他人调试过的超类。代码规模增长时，改动局部化、调试范围缩小（类把相关的名字和逻辑打包在一个命名空间里）。

·初学者误区：① 以为复用 = 复制粘贴 + 改改参数——那是"复制式复用"，继承才是"结构式复用"，一处修改全树受益；② 以为 OOP 代码一定要比过程式代码多——恰恰相反，写得好的类代码更少、改动更小；③ 把"方法手动传主体"（f(pat)）和继承式扩展混为一谈——前者复用函数，后者复用并定制行为，后者才是模块/函数给不了的能

力。

26.9.1 多态与类 (Polymorphism and classes)

英文原文

As an abstract example, suppose you're assigned the task of implementing an employee database application. As a Python OOP programmer, you might begin by coding a general superclass that defines default behaviors common to all the kinds of employees in your organization (code in this section is hypothetical and partial):

```
python class Employee: # General superclass
def computeSalary(self): ... # Common or default behaviors
def giveRaise(self): ...
def promote(self): ...
def retire(self): ...
```

Once you've coded this general behavior, you can specialize it for each specific kind of employee to reflect how the various types differ from the norm. That is, you can code subclasses that customize just the bits of behavior that vary per employee type; the rest of the employee types' behavior will be inherited from the more general class. For instance, if engineers have a unique salary computation rule (perhaps it's not hours times rate), you can replace just that one method in a subclass:

```
python class Engineer(Employee): # Specialized subclass
def computeSalary(self): ... # Something custom here
```

Because the computeSalary version here appears

lower in the class tree, it will replace (override) the general version in `Employee`. All other methods, though, are inherited from the superclass verbatim. You then create instances of the kinds of employee classes that the real employees belong to, to get the correct behavior:

```
python sue = Employee() # Default behavior bob = Employee() # Default behavior pat = Engineer() # Custom salary calculator
```

Notice that you can make instances of any class in a tree, not just the ones at the bottom—the class you make an instance from determines the level at which the attribute search will begin, and thus which versions of the methods it will employ (perhaps accidental). Ultimately, these three instance objects might wind up embedded in a larger container object—for instance, a list, dictionary, or an instance of another class—that represents a department or company using the composition idea mentioned at the start of this chapter. When you later ask for these employees' salaries, they will be computed according to the classes from which the objects were made, due to the principles of the inheritance search:

```
python company = [sue, bob, pat] # A composite object for emp in company: print(emp.computeSalary())
```

Run this emp's version: default or custom

This is yet another instance of the idea of polymorphism introduced in Chapter 4 and expanded in Chapter 16. Recall that polymorphism means that the meaning of an

operation depends on the object being operated on. That is, code shouldn't care about what an object is, only about what it does. Here, the method `computeSalary` is located by inheritance search in each object before it is called, per the object's class. The net effect is that we automatically run the correct version for the object being processed—default or custom. Trace the code to see why. In other applications, polymorphism might also be used to encapsulate (i.e., abstract away) interface differences. For example, a program that processes data streams might be coded to expect objects with input and output methods, without caring what those methods actually do:

```
python def processor(reader, converter, writer): while True: data = reader.read() if not data: break data = converter(data) writer.write(data)
```

By passing in instances of subclasses that specialize the required read and write method interfaces for various data sources, we can reuse the processor function for any data source we need to use, both now and in the future:

```
python class Reader: def other(self): ... # Default behavior and tools class FileReader(Reader): def read(self): ... # Read from a local file class SocketReader(Reader): def read(self): ... # Read from a network socket... and others ... processor(FileReader(...), converter, FileWriter(...)) processor(SocketReader(...), converter, FileWriter(...))
```

```
iter(...)) processor(FtpReader(...), converter, JsonWriter(...))
```

Moreover, because the internal implementations of those read and write methods have been factored into single locations, they can be changed without impacting code that uses them. The processor function might even be a class itself to allow the conversion logic of converter to be filled in by inheritance, and to allow readers and writers to be embedded by composition (you'll see how this works later in this part of the book).

中文翻译

作为一个抽象的例子，假设你被指派实现一个员工数据库应用程序。作为 Python OOP 程序员，你可能会从编写一个通用超类开始，它定义你组织中所有员工类型共有的默认行为（本节的代码是假设性的、不完整的）：`python class Employee: # 通用超类`
`def computeSalary(self): ... # 通用或默认行为`
`def giveRaise(self): ...`
`def promote(self): ...`
`def retire(self): ...`
编写完这个通用行为后，你可以针对每种具体员工类型进行专门化，以反映各类别与常规的差异。也就是说，你可以编写子类，只定制那些随员工类型而变的行为片段；其余员工行为从更通用的类继承。例如，如果工程师有独特的工资计算规则（也许不是工时乘以费率），你可以在子类中只替换这一个方法：`python class Engineer(Employee): # 专用子类`
`def computeSalary(self): ... # 这里有点自定义内容因为这里的 computeSalary 版本出现在类树中较低的位置，它将替换（覆盖）Employee 中的通用`

版本。不过，所有其他方法都原封不动地从超类继承。然后你创建真实员工所属的那些员工类的实例，以获得正确的行为：`python sue = Employee()` # 默认行为 `bob = Employee()` # 默认行为 `pat = Engineer()` # 自定义工资计算器请注意，你可以从树中的任何类创建实例，而不只是底部的那些——你用来创建实例的类决定了属性搜索将从哪个层级开始，因此也就决定了它采用哪个版本的方法（此处 "employ" 是 "采用/雇用" 双关，纯属巧合）。最终，这三个实例对象可能会被嵌入一个更大的容器对象中——例如列表、字典或另一个类的实例——利用本章开头提到的组合思想来表示一个部门或公司。稍后当你询问这些员工的工资时，它们将根据对象由哪个类创建而计算，这是继承搜索原理使然：`python company = [sue, bob, pat]` # 一个组合对象 `for emp in company: print(emp.computeSalary())` # 运行这个 emp 的版本：默认或自定义这是第 4 章引入、第 16 章扩展的多态 (polymorphism) 思想的又一个实例。回想一下，多态意味着一个运算的含义取决于被运算的对象。也就是说，代码不应该关心对象是什么 (is)，只应该关心它能做什么 (does)。这里，`computeSalary` 方法在每次调用前都会按对象所属的类，通过继承搜索在每个对象中定位。净效果是：我们会自动为正在处理的对象运行正确的版本——默认的或自定义的。追踪代码你就明白为什么了。在其他应用中，多态也可能被用来封装 (encapsulate，即抽象掉) 接口差异。例如，一个处理数据流的程序可以编写成 "期望拥有输入和输出方法的对象" 的形式，而不关心这些方法实际做什么：`python def processor(reader, converter, writer): while True: data = reader.read() if not data: break data = converter(data) writer.write(data)` 通过

传入针对各种数据源、专门化所需 read 和 write 方法接口的子类实例，我们就能从现在到未来，为任何需要使用的数据源复用 processor 函数：

```
python class Reader:
    def other(self): ... # 默认行为与工具
class FileReader(Reader):
    def read(self): ... # 从本地文件读取
class SocketReader(Reader):
    def read(self): ... # 从网络套接字读取...以及其他类型...
processor(FileReader(...), converter, FileWriter(...))
processor(SocketReader(...), converter, FileWriter(...))
processor(FtpReader(...), converter, JsonWriter(...))
```

此外，因为这些 read 和 write 方法的内部实现被分解到单一位置（factor into single locations），修改它们不会影响到使用它们的代码。processor 函数本身甚至也可以是一个类，以便让 converter 的转换逻辑通过继承来填充，并让 reader 和 writer 通过组合被嵌入（本书这一部分的后边你会看到这是怎么工作的）。

代码分析

员工数据库例子——继承 + 覆盖 + 组合 + 多态四合一

```
class Employee:
    #
    def computeSalary(self): ... #
    def giveRaise(self): ...
    def promote(self): ...
    def retire(self): ...

class Engineer(Employee):
    #
    def computeSalary(self): ... #

sue = Employee()
bob = Employee()
pat = Engineer()
```

```
company = [sue, bob, pat]           #  
for emp in company:  
    print(emp.computeSalary())      # emp
```

解释器如何处理：

1. 类定义阶段：Employee 的类对象先建好，其 dict 里有四个函数；Engineer 的类对象随后建好，其 bases 指向 Employee，其 dict 里只有 computeSalary 一个函数——只定制一个方法，其他三个自动“借”父类的。

2. 实例化阶段：pat = Engineer() 创建的实例，class 指向 Engineer；sue、bob 的 class 指向 Employee。于是同一次循环中：

- sue.computeSalary()：搜索路径 sue → Employee，命中 Employee.computeSalary；
- pat.computeSalary()：搜索路径 pat → Engineer，先在 Engineer.dict 命中自己的版本，根本到不了 Employee。

3. 这就是多态的运行时机：同一个名字 computeSalary、同一段循环代码，对不同的实例解析到不同的函数对象。调用方（for 循环）零感知、零分支——不需要 if type(emp) is Engineer: 之类的判断。

4. company 列表本身把三个实例装在一起，循环逐个分发——这体现组合（容器对象）与继承（行为定制）的协同：外层靠组合组织对象，内层靠继承决定每个对象的行为。

设计思想：作者点破了一个核心事实——“代码不应该关心对象是什么，只关心它能做什么”。这给了代码最大的灵

活性：今天只有 Engineer，明天加 Manager、Intern，循环代码一行不用改。多态把"变化"圈定在"新子类"这个最安全的位置。

processor 管道——接口抽象（鸭子类型雏形）

```
def processor(reader, converter, writer):    # read/...
    while True:
        data = reader.read()                #
        if not data: break                  #
        data = converter(data)              #
        writer.write(data)                  #

class Reader:
    def other(self): ...                    #
class FileReader(Reader):
    def read(self): ...                     #
class SocketReader(Reader):
    def read(self): ...                     #
# ... ..

processor(FileReader(...), converter, FileWriter(...))    # ...
processor(SocketReader(...), converter, FileWriter(...))  # ...
processor(FtpReader(...), converter, JsonWriter(...))     # FTP ...
```

解释器如何处理：processor 内部的 reader.read()、writer.write() 又是两次属性搜索——函数在运行时才解析名字，所以只要传入的对象在搜索链上有 read/write 方法，函数就能工作。解释器不检查、不声明参数类型，一切在运行时"按需取用"（这正是动态类型 + 多态的组合拳）。未来新增 FtpReader，只要实现 read，旧函数直接能用——这就是"面向接口编程"在 Python 中的自然形态（鸭子类型，duck typing）。

设计思想：read/write 的实现被"分解到单一位置"后，改文件读取的编码细节不影响 processor 骨架；反过来，processor 的流程改动也不影响各 Reader/Writer。耦合被降到最低——修改本地化，这正是本章反复强调的定制式编程的工程回报。

深度理解

·核心概念：多态 (polymorphism) = 操作的含义取决于操作的对象。同一段代码对不同对象表现不同行为，Python 中靠继承搜索自动分派 (dynamic dispatch)，无需任何 if-else。书中再次呼应第 4、16 章："代码关心对象能做什么 (does)，不关心它是什么 (is)"。

·底层实现：分派发生在每个 emp.computeSalary() 调用的字节码执行瞬间——LOADATTR 沿各实例自己的 class 链搜索，天然得到"各自版本"。所以多态不需要注册表、不需要虚拟函数表声明（对比 C++ vtable 的显式声明机制，Python 是隐式、纯运行时决定的）。

·设计原因：为什么"不判断类型"反而更安全？因为判断类型 (isinstance 分支) 会让代码与"已知类型集合"耦合——新增类型就得改代码。多态把类型集合开放：未来任意新增子类，旧代码自动兼容。这符合"开放—封闭原则"（对扩展开放、对修改封闭）。

·解决什么实际问题：① 员工数据库——按员工类型自动选择工资算法；② 数据管道——processor 一个函数通吃文件、网络、FTP、JSON 各种输入输出组合；③ 上层代码（循环、processor）与底层实现（各 Reader）解耦

，各自独立演化。

·初学者误区：① 用 `isinstance()` 硬编码分支代替多态——能跑，但每次新类型都要改分发代码，多态正是为此而生的替代品；② 以为多态必须靠继承——Python 里“只要行为像”（鸭子类型）即可，processor 甚至不检查参数是不是 Reader 的子类；③ 看到 `for emp in company` 时以为会“平均分配”默认方法——追一遍每个 emp 的 class 就能明白为何自动选对版本，追代码永远是最好的学习方式。

26.9.2 通过定制编程 (Programming by customization)

英文原文

Once you get used to programming this way (by software customization), you'll find that when it's time to write a new program, much of your work may already be done—your task largely becomes one of mixing together existing superclasses that already implement the behavior required by your program. For example, someone else might have written the Employee and Reader classes in this section's examples for use in completely different programs. If so, you get all of that person's code "for free." In fact, in many application domains, you can fetch or purchase collections of superclasses, known as frameworks, that implement common programming tasks as classes, ready to be mix

ed into your applications. These frameworks might provide database interfaces, testing protocols, GUI tool kits, and so on. With frameworks, you often simply code a subclass that fills in a handful of expected methods; the framework classes higher in the tree do most of the work for you. Programming in such an OOP world is just a matter of combining and specializing a already debugged code by writing subclasses of your own. Of course, it takes a while to learn how to leverage classes to achieve such OOP utopia. In practice, object-oriented work also entails substantial design work to fully realize the code reuse benefits of classes. To this end, programmers catalog common OOP structures, known as design patterns, to help with design choices. The actual code you write to do OOP in Python, though, is so simple that it will not in itself pose an additional obstacle to your OOP quest. To see why, you'll have to move on to Chapter 27.

中文翻译

一旦你习惯了这种编程方式（通过软件定制），你会发现，写一个新程序的时候，你的很多工作可能已经做完了——你的任务在很大程度上变成把现有的、已经实现你程序所需行为的超类混搭（mixing together）起来。例如，可能别人已经写了本节示例中的 `Employee` 和 `Reader` 类，用在完全不同的程序里。如果是这样，你就“免费”得到了那个人的全部代码。事实上，在许多应用领域，你可以获取或购买被称为框架（frameworks）的超类集合，它们把常见编程

任务实现为类，随时可以混入你的应用（applications）中。这些框架可能提供数据库接口、测试协议、GUI 工具包等等。使用框架时，你往往只需要编写一个子类，填上少数几个框架期望的方法；树中位置较高的框架类为你完成大部分工作。在这样的 OOP 世界里编程，不过就是通过编写自己的子类来组合（combining）和专门化（specializing）已经调试好的代码。当然，学会利用类实现这样的 OOP 乌托邦需要时间。在实践中，要充分发挥类的代码复用收益，面向对象工作还涉及大量的设计工作。为此，程序员们把常见的 OOP 结构编成目录，称为设计模式（design patterns），用来帮助设计决策。不过，你在 Python 中编写 OOP 的实际代码如此简单，它本身不会给您的 OOP 之旅增添额外的障碍。想知道为什么，你得继续读第 27 章。

深度理解

·核心概念：通过定制编程 = 组装现有超类 + 填补少量方法。写新程序的重心从“写代码”转移到“选对超类、补对子类”。这是框架（frameworks）和设计模式（design patterns）登上舞台的原因。

·底层实现：框架能“替你干活”的机制基础仍然是那一套：框架的超类方法通过继承被你的子类实例自动找到；框架内部调 self.某个方法() 时，多态保证调用到你的子类版本——这就是框架设计中最著名的“模板方法”（template method）模式：框架定义骨架流程，子类填实现。反向调用（框架代码调用你写的代码）也叫“控制反转”（IoC）。

·设计原因：为什么框架可行？因为“通用流程”（读取→转

换→写出) 极少变化, 而"具体实现" (从哪读、怎么转) 变化万千。把稳定部分放超类、易变部分留给子类, 改动成本降到最低。生活例子: 住酒店只需"入住—用餐—退房"走流程(框架), 至于吃什么(子类方法) 自己选。

·解决什么实际问题: Web 框架(如 Flask、Django)、GUI 工具包(Tkinter)、测试框架(unittest) 都是这个模式: 你只写"钩子"(hook) 方法, 其余框架代劳。设计模式则是前人踩坑沉淀的经验目录, 帮你少做错误设计决策。

·初学者误区: ① 以为框架是"黑魔法"——拆开看, 核心还是继承搜索 + 多态; ② 以为设计模式是"背名词"——本质是"在什么场景怎么组合类"的决策指南, 用多了自然理解; ③ 以为"OOP 乌托邦"立竿见影——作者坦承需要时间和设计经验, 别指望第一章就写出一流架构; ④ 把"定制"误解为"随心所欲改父类"——恰恰相反, 定制是不碰父类, 只新增子类。

26.10 本章小结 (Chapter Summary)

英文原文

We made an initial, abstract pass over classes and OOP in this chapter, taking in the big picture before we dive into syntax details. As we've seen, OOP is mostly about an argument named self, and a search for attributes in trees of linked objects called inheritance. Objects at the bottom of the tree inherit attributes fr

om objects higher up in the tree—a feature that enables us to program by customizing code, rather than changing it or starting from scratch. When used well, this model of programming can cut development time radically. The next chapter will begin to fill in the coding details behind the picture painted here. As we get deeper into Python classes, though, keep in mind that the OOP model in Python is very simple; as we've seen here, it's really just about looking up names in object trees and a special function argument. Before we move on, here's a quick quiz to review what we've covered here.

中文翻译

本章我们对类和 OOP 做了一次初步的、抽象层面的概览，在深入语法细节之前先把握全景。正如我们所看到的，OOP 主要就是关于一个名为 `self` 的参数，以及在链接对象树（称为继承）中搜索属性。树底部的对象从树中更高的对象继承属性——这一特性让我们能够通过定制代码来编程，而不是修改代码或从头开始。用得好时，这种编程模型能大幅缩短开发时间。下一章将开始填充本章所描绘图景背后的编码细节。不过，随着我们深入 Python 类，请记住：Python 中的 OOP 模型非常简单；正如我们在这里看到的，它其实就是关于在对象树中查找名字，以及一个特殊的函数参数。在继续之前，这里有一个快速测验，用来复习我们本章讲过的内容。

深度理解

·核心概念：一句话总结全章——OOP = self 参数 + 对象树属性搜索（继承）。没有第三条，没有隐藏魔法。

·底层实现："对象树"的物理形态 = class 指针 + bases 元组 + 各对象自己的 dict；"查找" = 沿链逐一比对字典键。全书所有类代码最终都跑在这套简单机制上。

·设计原因：作者刻意把模型压到极简，是为了让"定制式编程"成为可推理的行为：树的形状你亲手搭，搜索顺序确定且可预测，覆盖规则简单，因此程序行为可预期、可调试。

·解决什么实际问题：把 OOP 从"玄学"还原为"机制"——遇到任何奇怪的对象行为，先问两个问题：这个属性在树上哪个位置？这个调用传的 self 是谁？答案到手，问题基本解决。

·初学者误区：① 本章结束时以为已经掌握全部语法——这只是全景图，语法细节（第 27-30 章）尚未展开；② 轻视"简单"——越是简单的机制越要抠透，它是后面所有高级特性（描述符、元类、MRO）的基石。

26.11 测验：测试你的知识 (Test Your Knowledge: Quiz)

英文原文

1. What is the main point of OOP in Python? 2. Where

e does an inheritance search look for an attribute? 3. What is the difference between a class object and an instance object? 4. Why is the first argument in a class's method function special? 5. What is the init method used for? 6. How do you create a class instance? 7. How do you create a class? 8. How do you specify a class's superclasses?

中文翻译

1. Python 中 OOP 的主要目的是什么? 2. 继承搜索在哪里查找属性? 3. 类对象和实例对象有什么区别? 4. 为什么类的方法函数中的第一个参数很特殊? 5. init 方法是做什么用的? 6. 如何创建一个类实例? 7. 如何创建一个类? 8. 如何指定一个类的超类?

深度理解 (答题提示)

这 8 个问题是全章的“检查表”: 第 1 题考“为什么”(代码复用), 第 2-3 题考“机制”(树搜索、类 vs 实例), 第 4 题考“self”(隐含主体), 第 5-8 题考“语法钩子”(init、类调用、class 语句、括号列表超类)。如果你能不看答案把 8 题全部写清楚, 本章就过关了。

建议把每题用自己的话讲一遍(就像讲给别人听), 比反复读更有效——这是检验理解度的黄金法则(费曼学习法)。

26.12 答案：测试你的知识 (Test Your Knowledge: Answers)

英文原文

1. OOP is about code reuse—you factor code to minimize redundancy, and program by customizing what already exists instead of changing code in place or starting from scratch. 2. An inheritance search looks for an attribute first in the instance object, then in the class the instance was created from, then in all higher superclasses, progressing from the bottom to the top of the object tree, and from left to right (normally). The search stops at the first place the attribute is found. Because the lowest version of a name found along the way wins, class hierarchies naturally support customization by extension in new subclasses. 3. Both class and instance objects are namespaces—packages of variables that appear as attributes. The main difference between them is that classes are a kind of factory for creating multiple instances. Classes also support operator-overloading methods, which instances inherit, and treat any functions nested in the class as methods for processing instances. 4. The first argument in a class's method function is special because it always receives the instance object that is the implied subject of the method call. It's usually called `self` by convention. Because method functions always have this impl

ied subject—and object context—by default, we say they are "object-oriented" (i.e., designed to process or change objects).

5. If the `init` method is coded or inherited in a class, Python calls it automatically each time an instance of that class is created. It's known as the constructor method; it is passed the new instance implicitly, as well as any arguments passed explicitly to the class name. It's also the most commonly used operator-overloading method. If no `init` method is present, instances simply begin life as empty namespaces.

6. You create a class instance by calling the class name as though it were a function; any arguments passed into the class name show up as arguments two and beyond in the `init` constructor method (if there is one). The new instance remembers the class it was created from for inheritance purposes.

7. You create a class by running a class statement; like function definitions, these statements normally run when the enclosing module file is imported (more on this in the next chapter).

8. You specify a class's superclasses by listing them in parentheses in the class statement, after the new class's name. The left-to-right order in which the classes are listed in the parentheses gives the left-to-right inheritance search order in the class tree.

中文翻译

1. OOP 的核心是代码复用 (reuse) ——你把代码分解以最小化冗余，并通过定制已有的东西来编程，而不是就地修

改代码或从头开始。2. 继承搜索先在实例对象中查找属性，然后在创建该实例的类中查找，再在所有更高的超类中查找，沿对象树自底向上、并（通常）从左到右推进。搜索在第一次找到该属性的地方停止。因为沿途找到的名字中最低的版本获胜，类层次天然支持通过在新子类中扩展来实现定制。3. 类对象和实例对象都是命名空间——表现为属性的变量包裹体。它们的主要区别在于：类是创建多个实例的一种工厂。类还支持运算符重载方法（实例会继承它们），并把类内嵌套的任何函数都当作处理实例的方法。4. 类的方法函数中第一个参数很特殊，因为它总是接收作为方法调用隐含主体的实例对象。按惯例它通常叫 `self`。因为方法函数默认总有这个隐含主体——即对象上下文——所以我们说它们是“面向对象的”（即设计用来处理或改变对象的）。5. 如果 `init` 方法在类中被定义或被继承，Python 每次创建该类实例时都会自动调用它。它被称为构造器（`constructor`）方法；新实例被隐式传给它，显式传给类名的任何参数也会传给它。它也是最常用的运算符重载方法。如果没有 `init` 方法，实例就以空命名空间的身份开始生命。6. 你通过把类名当作函数来调用来创建类实例；传给类名的任何参数都会作为 `init` 构造器方法（如果有的话）的第二个及之后的参数出现。新实例会记住它由哪个类创建，以备继承之用。7. 你通过运行 `class` 语句来创建类；和函数定义一样，这些语句通常在所在模块文件被导入时运行（下一章会详细讨论）。8. 你通过在 `class` 语句中、新类名之后的括号里列出超类来指定类的超类。括号中列出的类从左到右的顺序，就给出了类树中从左到右的继承搜索顺序。

深度理解

·答案 2 是全章最重要的机制总结：先实例、后类、再超类，自底向上、从左到右，首次命中即停，最低版本获胜。

请把它当口诀背下来。

·答案 3 提醒"类与实例都是命名空间"——这是贯穿整本书的对象观：任何东西都不过是"可挂名字的包"。

·答案 5 强调 init 的可选性与自动性：没有它不报错（空实例），有它必自动调（每次创建）。这个"静默降级"是陷阱也是便利，务必牢记。

·答案 7 与模块机制挂钩：类语句在 import 时执行，所以"导入模块"会连带创建类对象——这也意味着类的创建时机是确定的、可预测的。

26.13 脚注 (Footnotes)

英文原文

1 In other literature and circles, you may also occasionally see the terms base classes and derived classes used to describe superclasses and subclasses, respectively. Most Python people and this book tend to use the latter terms. 2 If you've ever used C++ or Java, you'll recognize that Python's self is much like these languages' this pointer/reference, but self is always explicit in both headers and bodies of Python methods to make attribute accesses more obvious: a name has fe

wer possible meanings to consider, if it cannot be magically associated with a hidden object. Explicit is generally better.

中文翻译

1 在其他文献和圈子里，你可能偶尔会看到用基类 (base classes) 和派生类 (derived classes) 分别描述超类和子类。大多数 Python 从业者和本书倾向于使用后一组术语 (即 superclass/subclass)。2 如果你用过 C++ 或 Java，你会认出 Python 的 self 很像这些语言中的 this 指针/引用，但 self 在 Python 方法的头部和函数体中都始终是显式的，这让属性访问更加一目了然：如果名字不能神秘地与某个隐藏对象关联，那么需要考虑的"可能含义"就更少。显式通常更好。

深度理解

·脚注 1：术语地图——superclass = base class (基类)，subclass = derived class (派生类)。读旧书、旧代码时两套术语都要认识，但写代码、交流时按本书惯例用 superclass/subclass。

·脚注 2：Python 与 C++/Java 在 OOP 上的一个关键哲学分歧——self 显式 vs this 隐式。隐式 this 省打字，但读代码时一个裸名字可能是局部变量、成员变量或全局变量；显式 self.x 一眼可知是对象属性。Python 选了"更啰嗦但更清楚"。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

- 分层架构：Django/Flask 等 Web 框架中，View 基类 + 子类重写 get()/post()，正是本章"通用超类 + 子类填补"模式的教科书级应用；User、Order 等模型类则示范了"类 = 记录的程序"。
- GUI 编程：Tkinter/PyQt 中，窗口类是容器（组合），Button 等控件类是组件；自定义控件就是写 Widget 的子类（继承）。
- 插件系统：定义抽象基类（ABC，第 30 章）规定接口，第三方插件以子类形式接入，宿主程序用多态自动调用——零改动扩展生态。
- 测试框架：unittest 的 TestCase 基类定义 setUp/test 骨架，你写子类填用例——框架替你跑"骨架流程"。

与 C++/Java 的区别

维度	Python	C++/Java
self/this	self 显式出现在参数列表与函数体	this 隐式可用
属性查找	运行时沿类树动态搜索	编译期静态解析（虚函数除外）
类型声明	动态类型，无需声明接口	必须声明类与继承关系
多态实现	纯运行时按对象自动分派	需要虚函数/vtable 机制声明

构造函数	init (初始化) + new (分配) 分离	构造函数合一
属性创建	首次赋值即创建, 无需预声明	必须在类体声明成员变量
多重继承	原生支持 (MRO 解决冲突)	Java 不支持 (用接口), C++ 支持但复杂

Python 发展历史背景

· class 语句自 Python 1.0 (1994 年) 就存在, 且核心语义 (实例属性在类树底部、搜索自底向上) 至今 30 年没有改变——这正是本章内容的长期稳定性所在。

· 早期 Python 的"类"受 Smalltalk 影响, 强调"消息传递"与"动态分派"; Guido 刻意让类机制比 C++ 简单: 没有 private/protected 关键字 (约定用下划线)、没有抽象类声明 (用 raise NotImplementedError 模拟), 一切靠约定与运行时。

· 现代演进 (Python 2.2+ 统一类型模型) 引入"新式类", 类自身也成为 type 的实例 (元类出现); Python 3 统一为新式类, mro、super() 等现代工具才成为常态——第 28 章会系统讲解。

高级开发者需要掌握的相关知识

· mro 与 C3 线性化: 多重继承的真实搜索顺序比"从左到右"更精细 (第 28 章), 菱形继承时必须掌握。

· 鸭子类型与 ABC: 多态的边界——何时用鸭子类型"够用就好", 何时用 abc.ABC 强制接口 (第 30 章)。

· 描述符与属性协议: @property、getattr 等属性访问钩子, 是"属性查找"机制的现代扩展 (第 31 章)。

- `super()` 协作式继承：多继承下正确调用父类方法的方式，不是 `Parent.init(self)` 直呼其名。
- 现代替代品：dataclasses（数据类）、attrs 库让"写记录型类"大幅简化，但理解本章的机制仍是正确使用它们的前提。

学习建议 (Learning Advice)

- 重要程度：★★★★★ (5/5) 。OOP 是 Python 的三大支柱之一（另两者：面向过程与函数式），后续第 27-31 章全部建立在本章的概念之上；几乎所有主流库（Django、pandas、matplotlib）都以类为组织单位。
- 应该掌握到什么程度：
- 能不看资料复述 8 个测验问题及其答案；
- 能手绘一棵类树（实例/类/超类）并口算出任意 `obj.attr` 的搜索路径；
- 能解释 `I1.setname('sue')` 与 `C1.setname(I1,'sue')` 的关系，以及 `init` 何时被自动调用；
- 记住两句口诀："读走链、写本地"（属性查找与赋值的区别）；"实例是记录，类是程序"。
- 后续应该学习哪些相关内容：
- 第 27 章：class 语句的完整语法细节（类体、命名空间、执行时机）；
- 第 28 章：继承、`super`、MRO——把本章的"树"讲到精确；

- 第 29 章：运算符重载全家族（str、getitem、call.....）
；
- 第 30 章：抽象类与鸭子类型；第 31 章：属性访问协议与描述符；
- 实践建议：把学到的树搜索机制画在纸上调试一次真实的 `AttributeError`——这是把"机制"变成"手感"的最快路径。